Neural Networks

Dr Aishwarya S., Assistant Professor of Mathematics, MIT, Manipal

Artificial Neural Network

- Mathematical model for learning inspired by biological neural networks.

Artificial Neural Network

- A **neural network** is a **mathematical model that learns a function** from examples.

- It takes **input features** → transforms them through a sequence of layers → produces an **output** (**prediction**).

Motivation: From Linear Models to Neural Networks

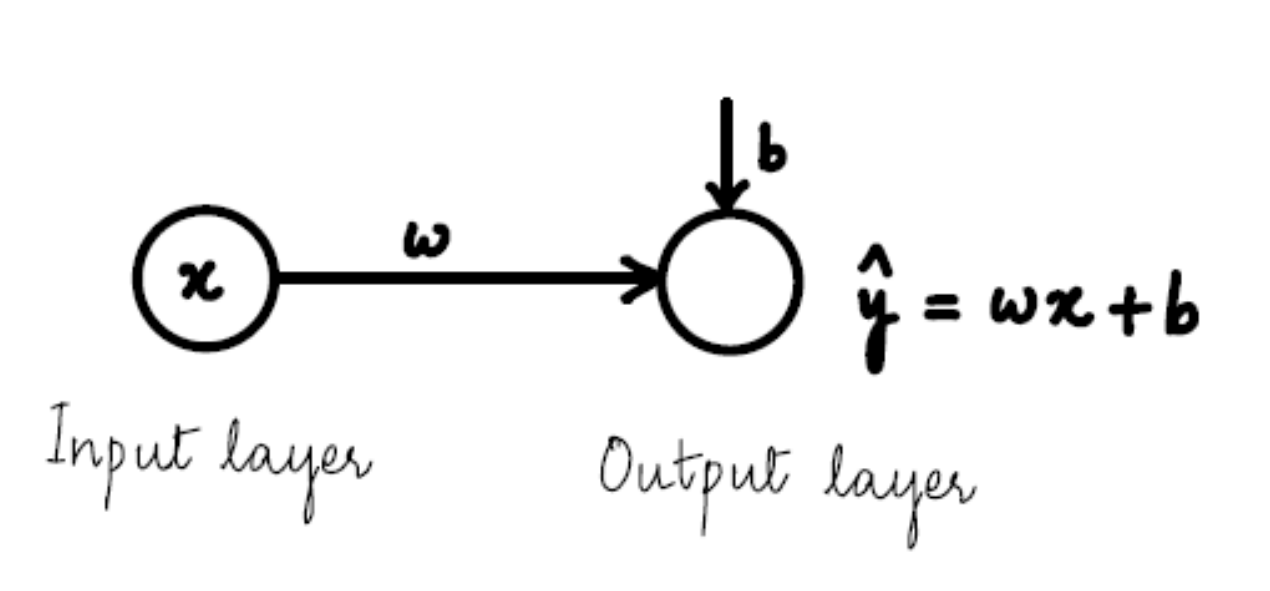
Linear Regression: $y = w^T x + b$.

Limitation: Captures only linear patterns in the data.

Data is often more complex (like images, speech, etc.) and needs more complex functions to model.

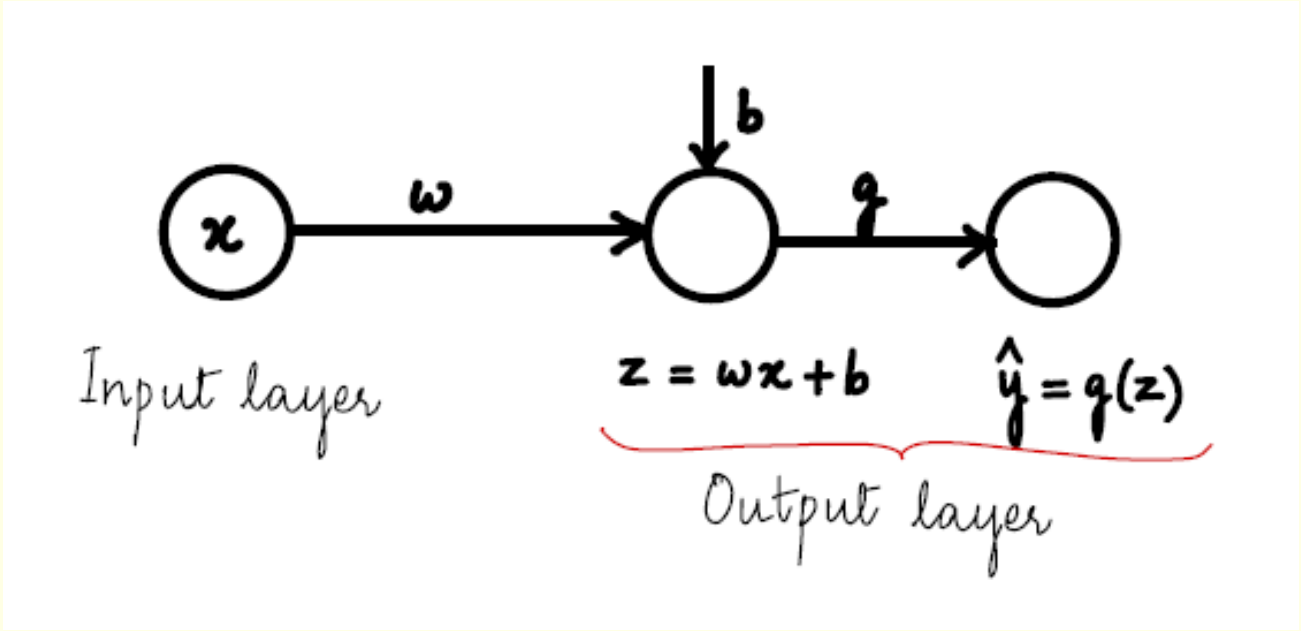
Neural networks compose many simple linear maps with non-linearities.

A single layer neural network with a single input feature

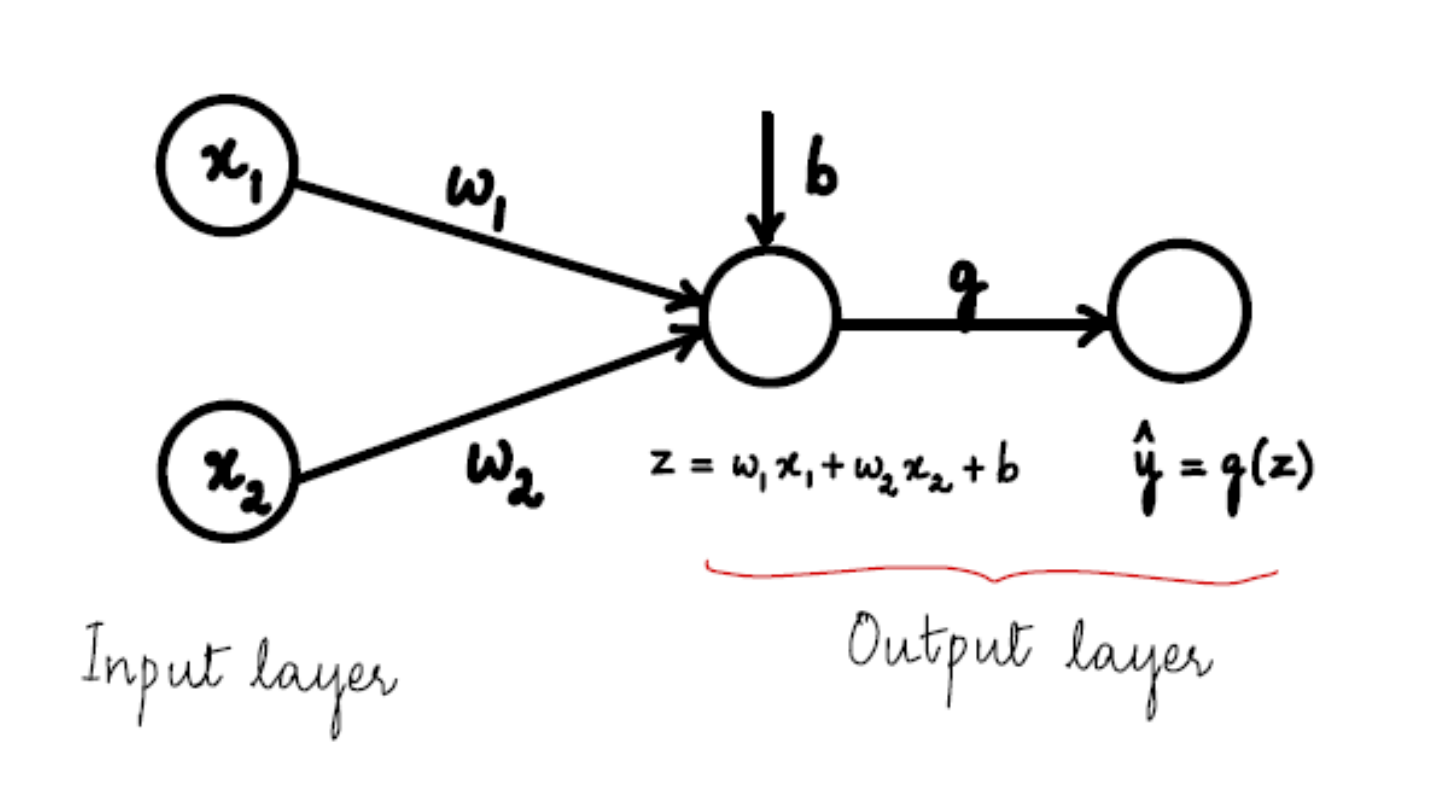


Note: The input layer is not counted under layers in a neural network.

A single layer neural network with a single input feature and an activation function at the output layer



A single layer neural network with two input features and an activation function at the output layer

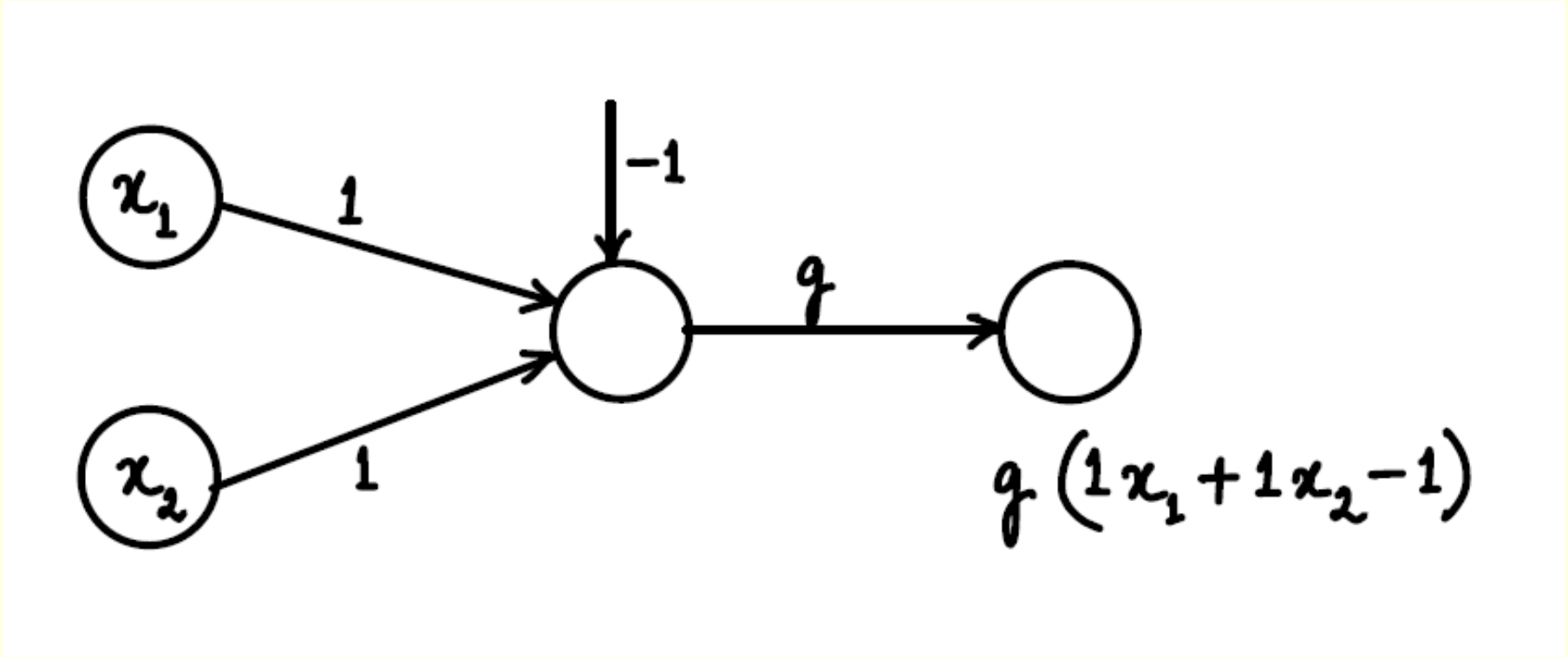


Learning OR Function using a Neural Network

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Take $g(z)$ as the step function:

$$g(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

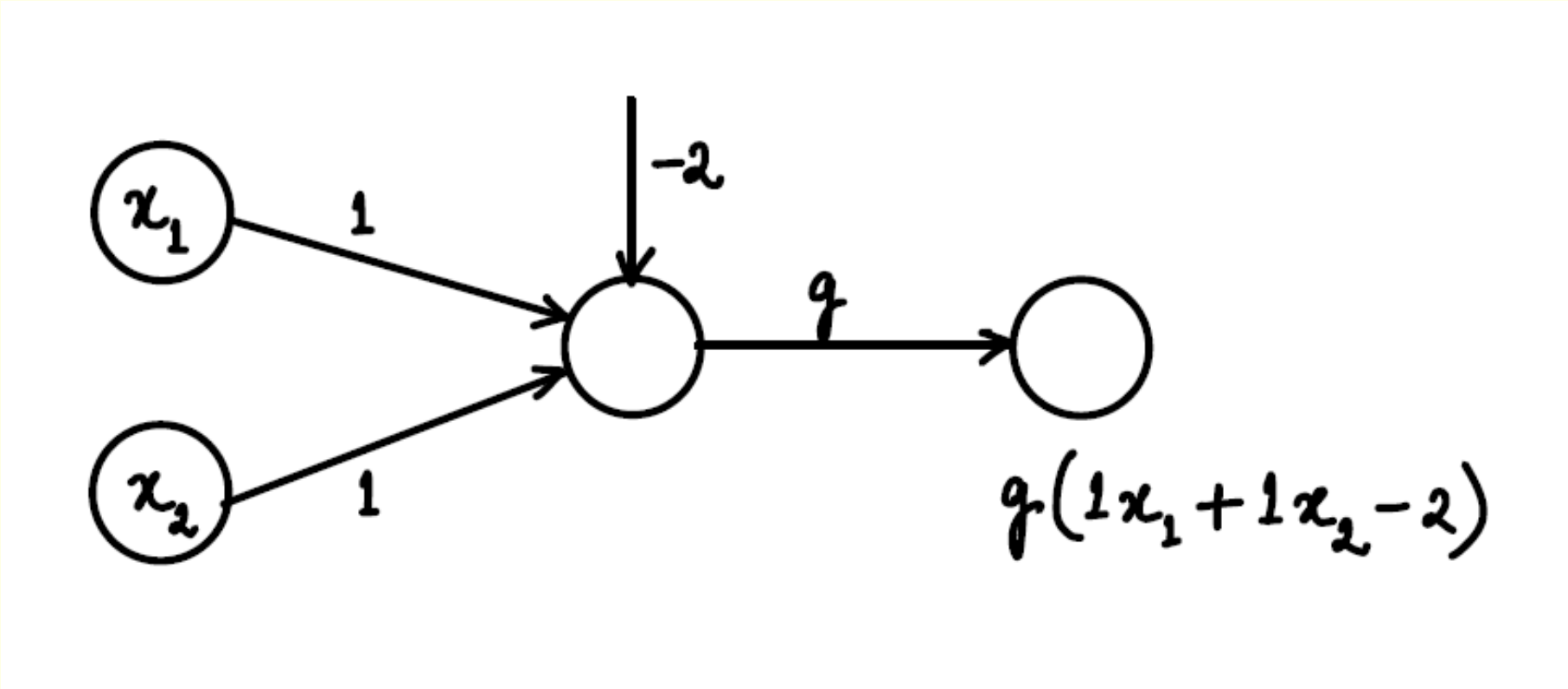


Learning AND Function using a Neural Network

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Take $g(z)$ as the step function:

$$g(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$



- For neural networks used for binary classification tasks, the sigmoid function

$$g(z) = \frac{1}{1+e^{-z}}$$

is usually used at the output layer to get the required probabilities as the outputs.

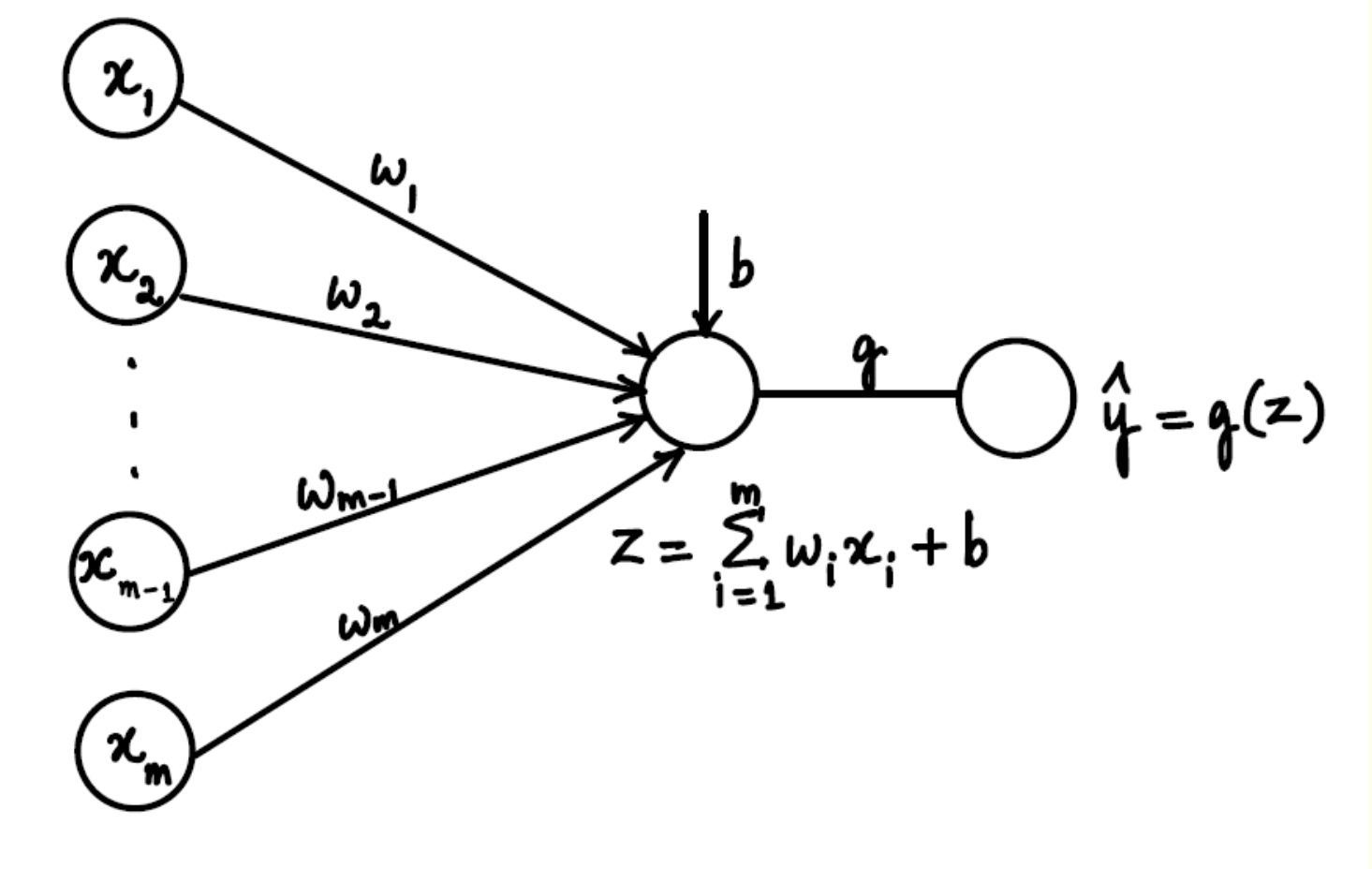
- For neural networks used for multiclass classification tasks, the softmax function

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

is usually used at the output layer to get the required probability distribution as the output.

- For neural networks used for regression tasks, the identity function (sometimes called the "linear activation function") is used at the output layer.

In general, a single layer neural network with m input features is as follows:



Training a neural network (with a single layer)

The weights (parameters) in the neural network are learnt using the **Gradient Descent Algorithm**.

The following is a general description of the Gradient Descent Algorithm for linear regression:

1. Start with a random choice of weights (model parameters).
2. Repeat until convergence:
    (i) Compute the gradient of the loss function with the current weights.
    (ii) Update the weights in the direction that reduces the loss.

Vectorized Gradient Descent Algorithm for Linear Regression

Suppose there are n training examples, each having m input features. Let

$$X \in \mathbb{R}^{n \times m}, \quad w = \begin{bmatrix} w_1 & w_2 & \cdots & w_m \end{bmatrix}^T \in \mathbb{R}^{m \times 1}, \quad y \in \mathbb{R}^{n \times 1},$$

where X is the input matrix, w is the vector of weights, y is the target vector, and $b \in \mathbb{R}$ is the bias. Let $\mathbf{1}_n$ denote the $n \times 1$ vector of ones.

Prediction and Cost Function

The predictions for all n training examples are computed simultaneously as

$$\hat{y} = Xw + b\mathbf{1}_n.$$

Using the mean squared error, the cost function is

$$J(w, b) = \frac{1}{2n} \|\hat{y} - y\|_2^2 = \frac{1}{2n} (\hat{y} - y)^T (\hat{y} - y).$$

Gradient Vectors

The gradient of J with respect to the weight vector w is

$$\nabla_w J = \frac{1}{n} X^T (\hat{y} - y).$$

Thus,

$$\nabla_w J = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \\ \vdots \\ \frac{\partial J}{\partial w_m} \end{bmatrix},$$

so it contains the partial derivatives with respect to all the weights. The derivative with respect to the bias is

$$\frac{\partial J}{\partial b} = \frac{1}{n} \mathbf{1}_n^T (\hat{y} - y) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i).$$

Vectorized Gradient Descent Algorithm

1. Initialize w and b.

2. For each iteration, compute

$$\hat{y} = Xw + b1_n, \qquad \nabla_w J = \frac{1}{n} X^T(\hat{y} - y), \qquad \frac{\partial J}{\partial b} = \frac{1}{n} 1_n^T(\hat{y} - y).$$

3. Update the parameters simultaneously:

$$w \leftarrow w - \alpha \nabla_w J, \qquad b \leftarrow b - \alpha \frac{\partial J}{\partial b},$$

   where $\alpha > 0$ is the learning rate.

4. Repeat until the chosen number of iterations is reached or the cost converges.

The following variants of gradient descent are often used for faster training:

- Stochastic gradient descent
- Mini-batch gradient descent

However, a simple (linear) model is often not sufficient to learn the patterns in the real-world input data well. Hence multi-layer neural networks are used.

Multi-Layer Neural Network

A multi-layer neural network is an artificial neural network which has an input layer, an output layer and **at least one** intermediate layer, called the "**hidden layer**".

- Note that if every layer of a multi-layer neural network learns only a linear function of its input, then neural network can finally learn only a linear function even with numerous layers.

Suppose we have two layers, both with linear maps.

Layer 1: $z_1 = w_1 x + b_1$

Layer 2:
$$\begin{aligned} z_2 &= w_2 z_1 + b_2 \\ &= w_2(w_1 x + b_1) + b_2 \\ &= (w_2 w_1)x + (b_1 + b_2) \\ &= w_3 x + b_3, \text{ where } w_3 = w_2 w_1 \text{ and } b_3 = b_1 + b_2. \end{aligned}$$

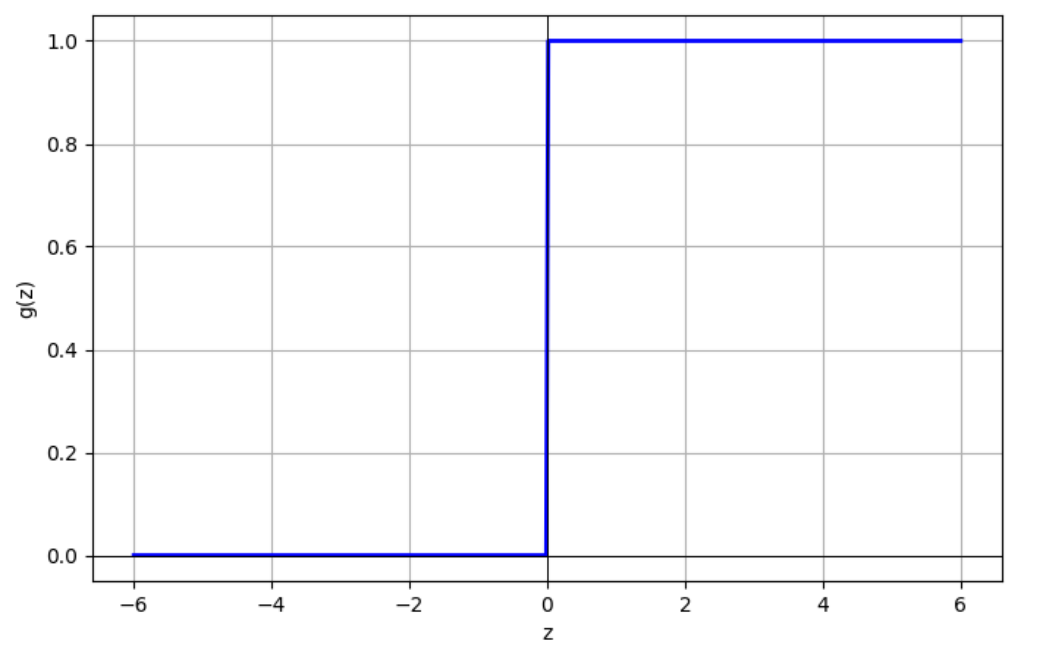
This shows the need to introduce non-linearity in the network.

The non-linearity required for the model is introduced by applying some non-linear functions in the intermediate layers of the neural network. These functions are called **Activation Functions.**

Some common Activation Functions

Step function: (used in early neural networks, but rarely in modern practice)

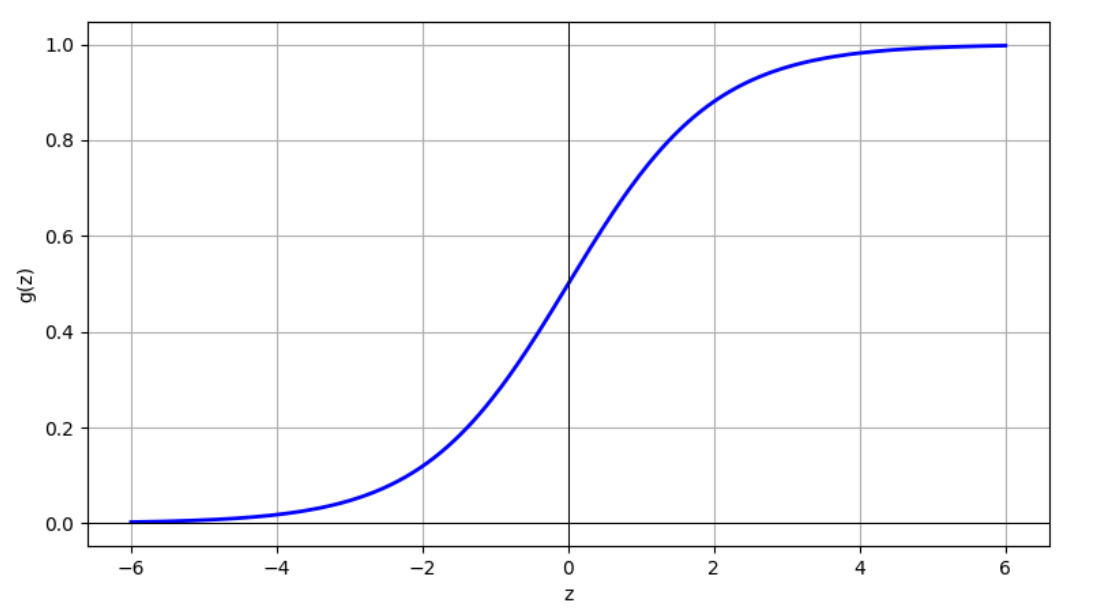
$$g(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$



Sigmoid function:

- Mostly used in output layers of neural networks for binary classification because its range is (0, 1) and the output can be interpreted as a probability.

$$g(z) = \frac{1}{1 + e^{-z}}$$



Softmax function:

- Mostly used in output layers of neural networks for multiclass classification because it outputs a probability distribution.

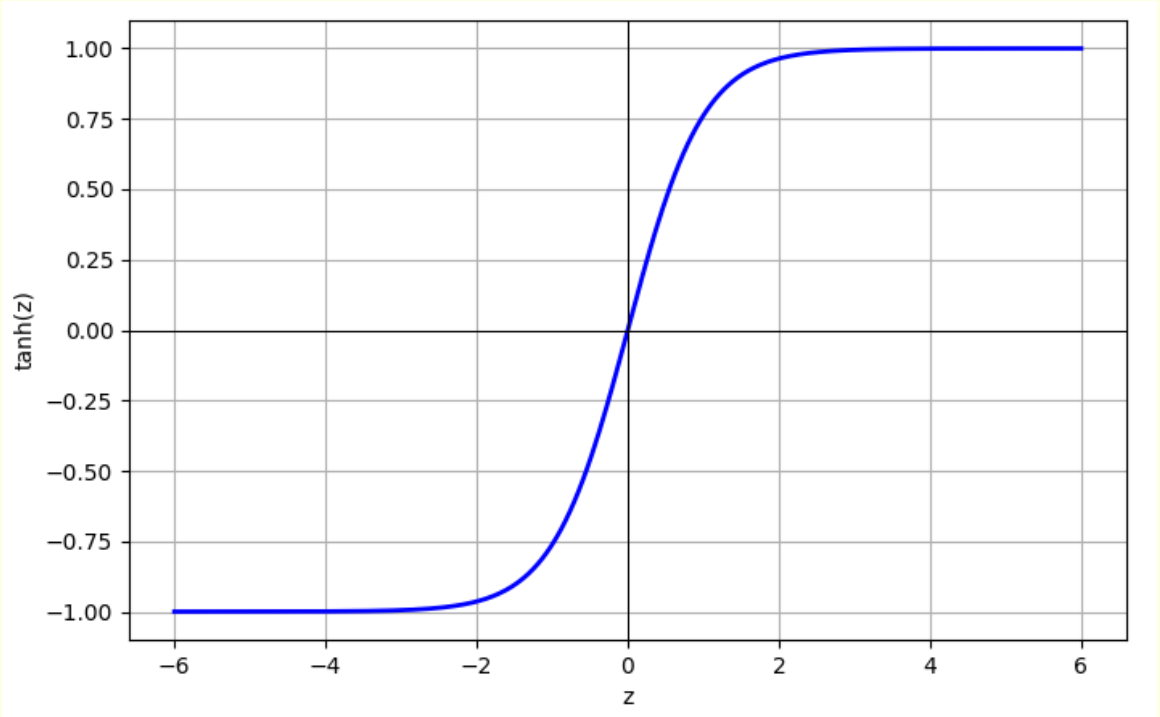
- Given a vector of real-valued inputs $(z_1,\ z_2,\ \ldots, z_k)$,

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}.$$

Tanh function:

$$tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

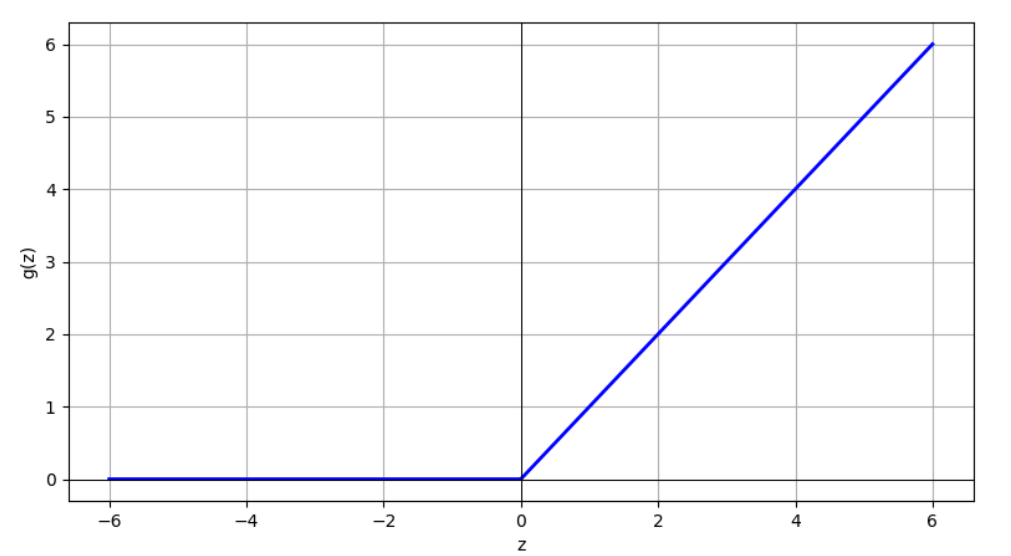
$tanh$ has the range (-1, 1).



ReLU (Rectified Linear Unit) function:

- Mostly used in the hidden layers of modern neural networks because it is simple and efficient.

$$g(z) = \max\{0, z\} = \begin{cases} z, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

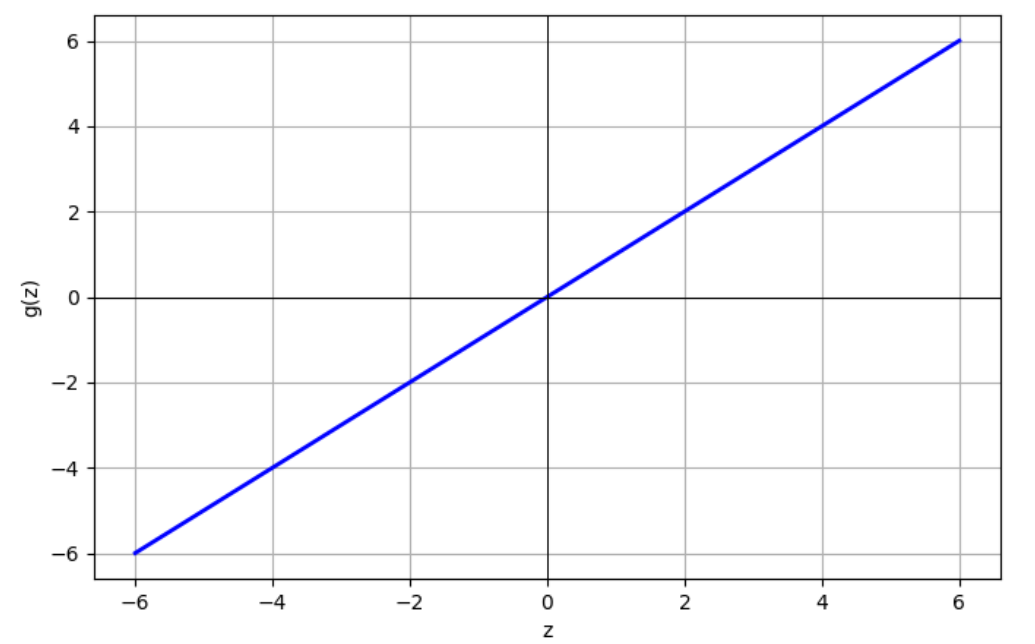


Linear activation function (Identity function):

- Mostly used in the output layer of neural networks for regression problems.

$$g(z) = z$$

(No non-linearity applied.)



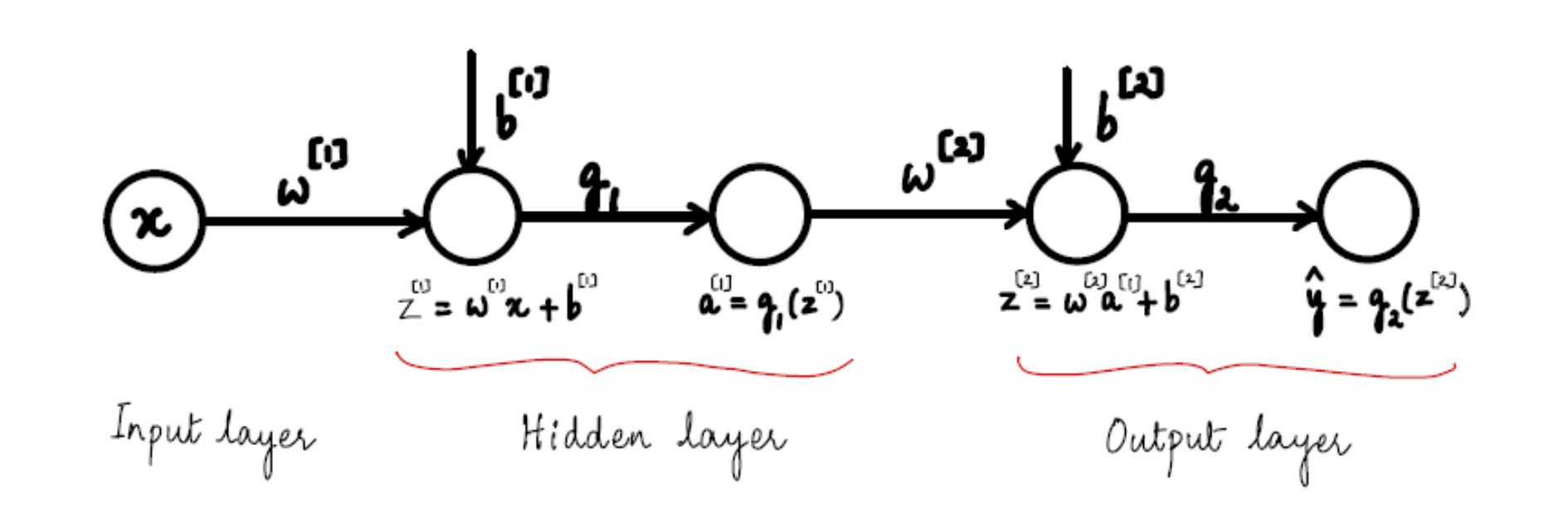
Universal Approximation Theorem

A feedforward neural network with a single hidden layer that uses a non-polynomial activation function, can approximate any continuous function to an arbitrary degree of accuracy.

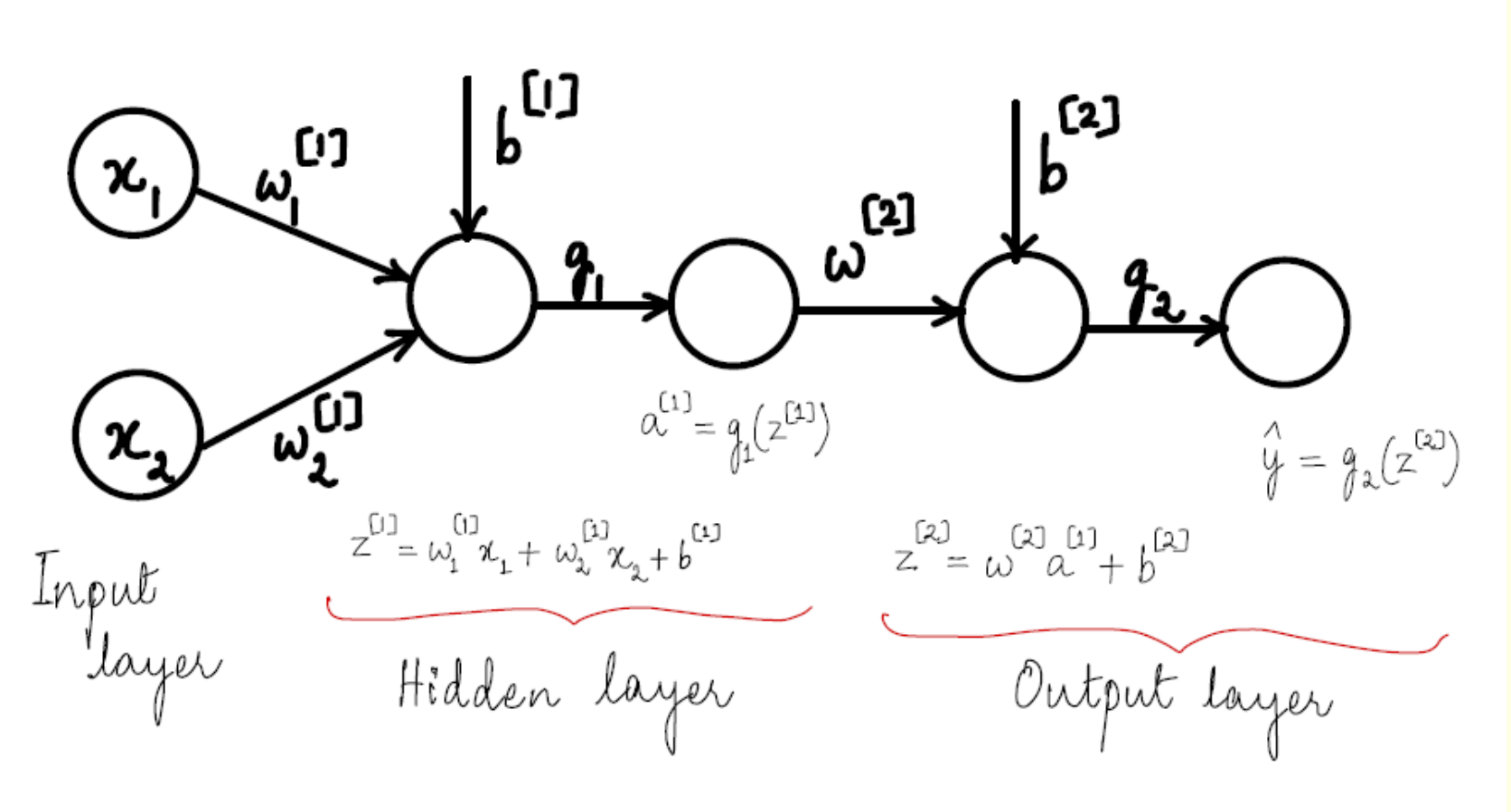
Universality is achieved by increasing the number of neurons in the hidden layer, making the network “wider”.

- Other versions of the theorem show that universality can also be achieved by keeping the width of the network fixed and increasing the number of layers, making it “deeper”.

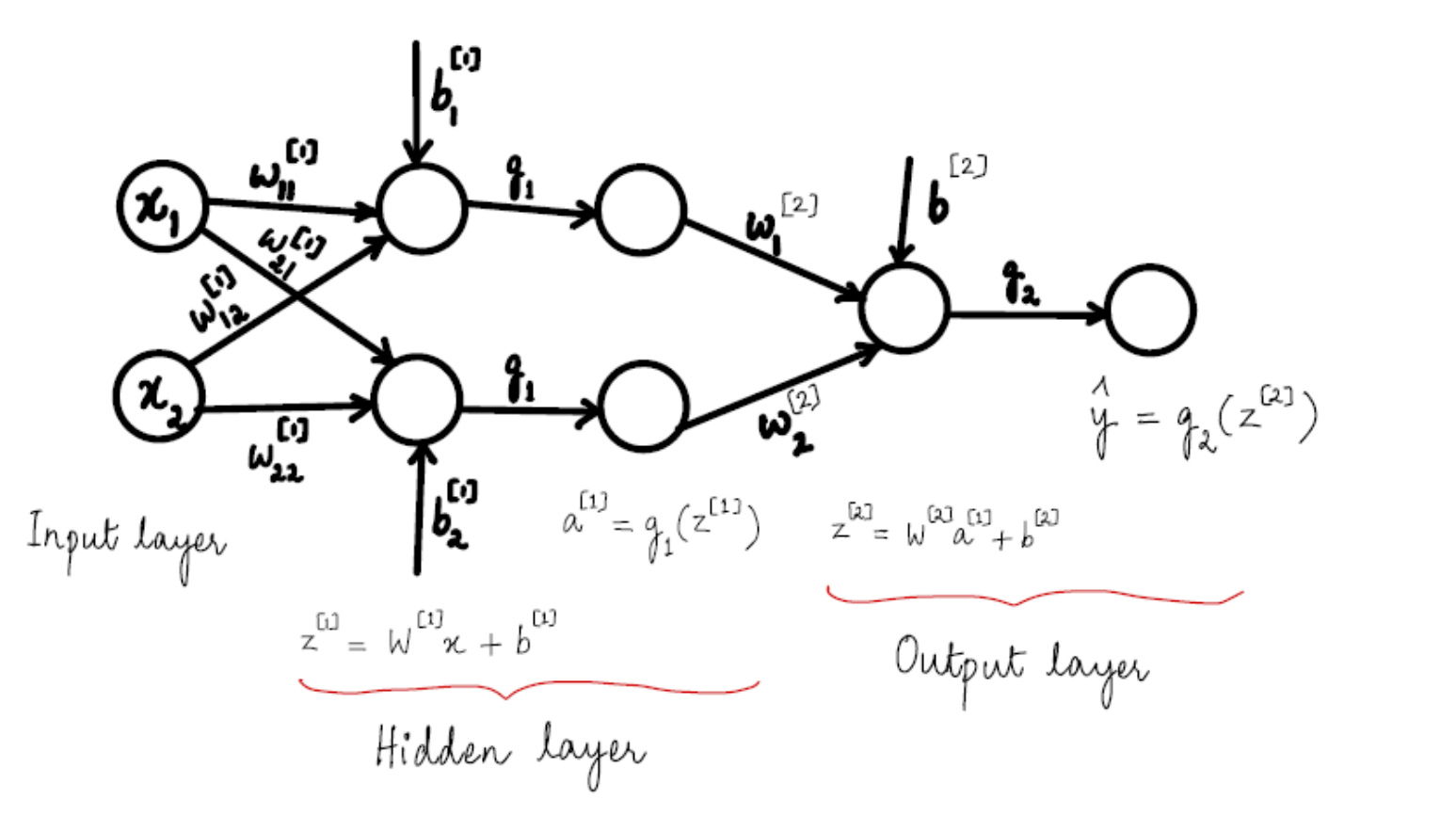
A two-layer neural network with a single input feature



A two-layer neural network with two input features



A two-layer neural network with two input features, with multiple nodes (neurons) in the hidden layer

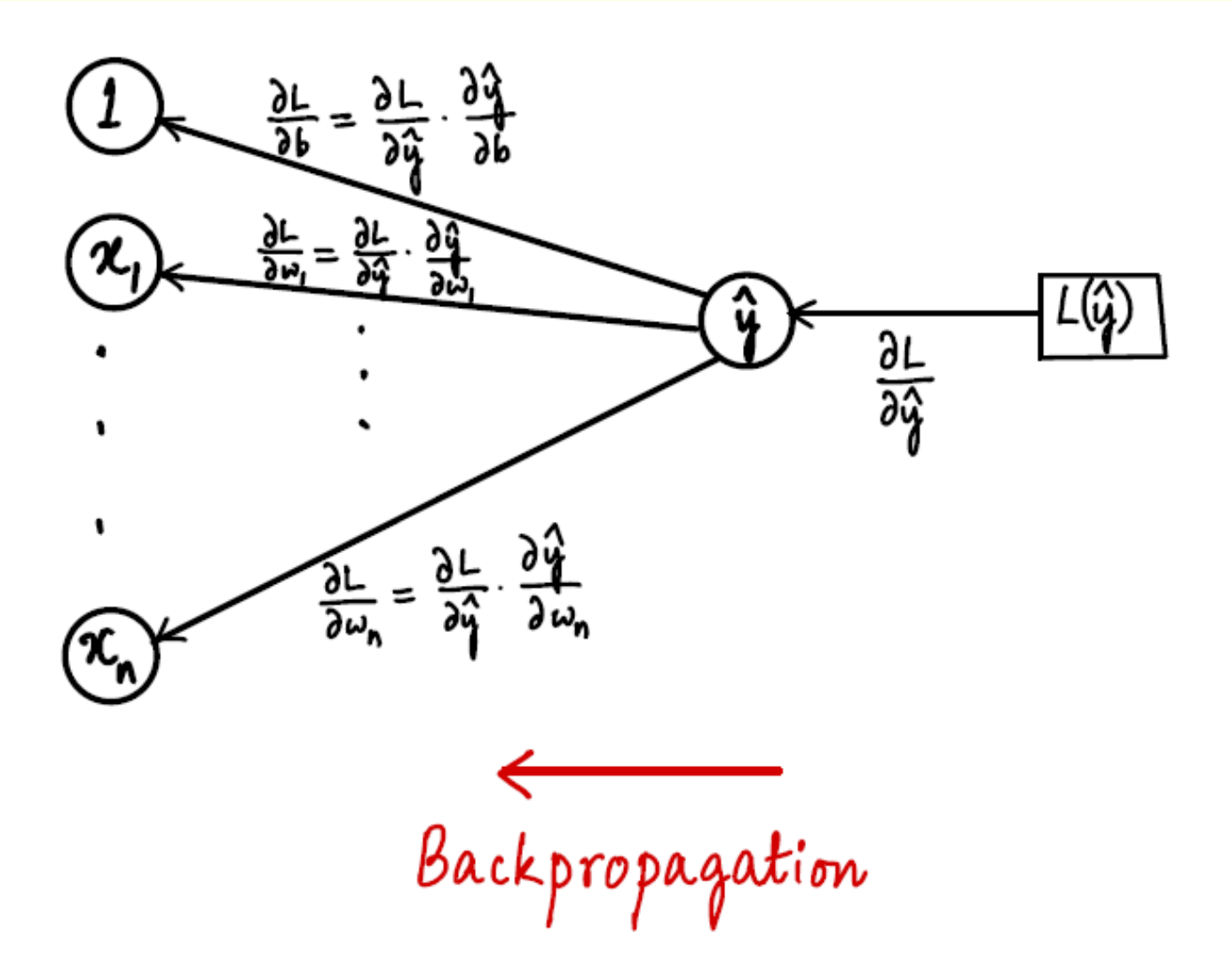


$$W^{[1]} = \begin{bmatrix} {w_{11}}^{[1]} & {w_{12}}^{[1]} \\ {w_{21}}^{[1]} & {w_{22}}^{[1]} \end{bmatrix}, \qquad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad b^{[1]} = \begin{bmatrix} {b_1}^{[1]} \\ {b_2}^{[1]} \end{bmatrix},$$

$$z^{[1]} = \begin{bmatrix} {z_1}^{[1]} \\ {z_2}^{[1]} \end{bmatrix}, \qquad a^{[1]} = \begin{bmatrix} g_1({z_1}^{[1]}) \\ g_1({z_2}^{[1]}) \end{bmatrix},$$

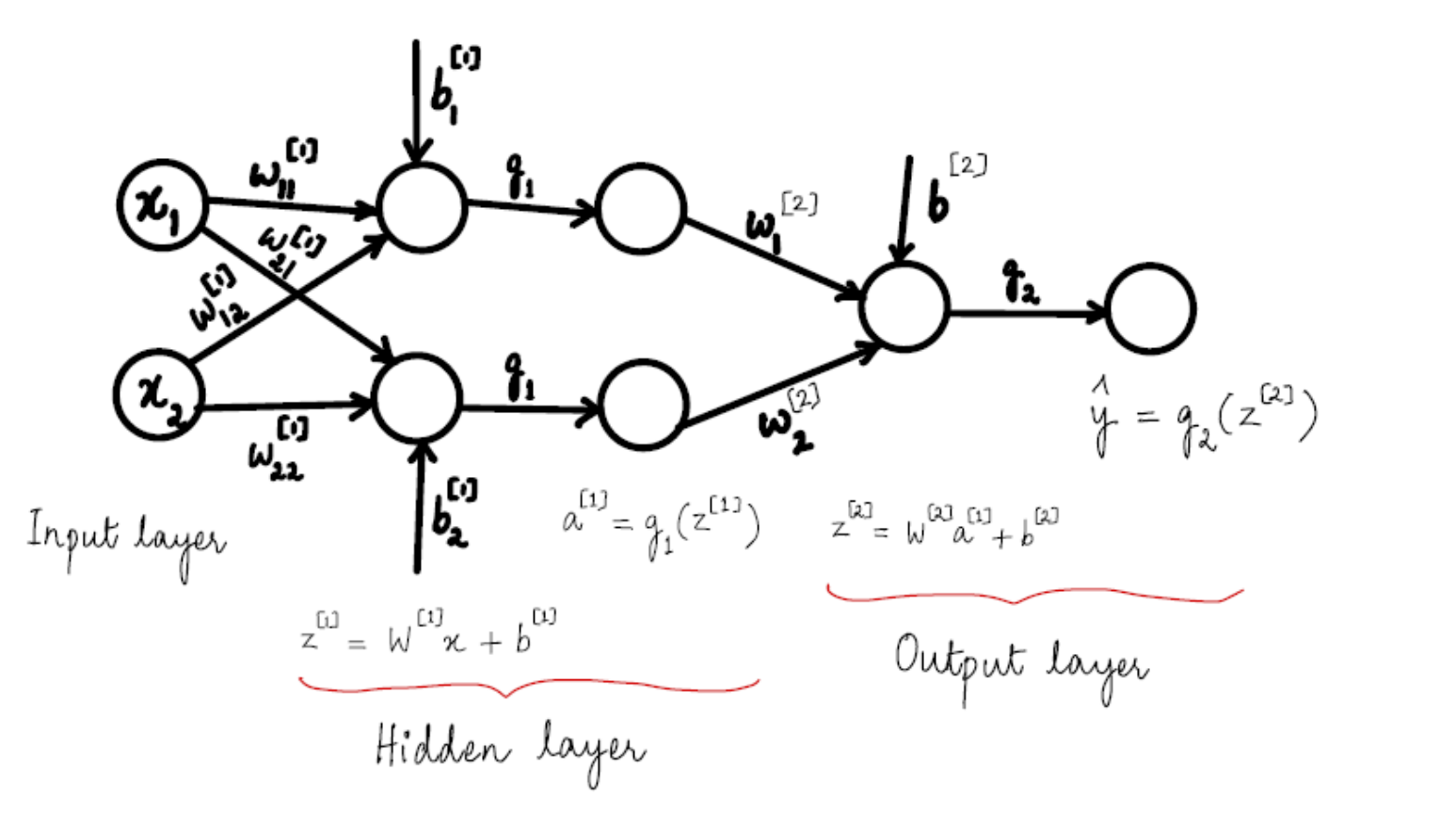
$$W^{[2]} = \begin{bmatrix} {w_1}^{[2]} & {w_2}^{[2]} \end{bmatrix}.$$

Training a multi-layer neural network by Backpropagation of Errors



Computation of $\frac{\partial J}{\partial w_{11}^{[1]}}$ in the following neural network:

(Consider $J = (\hat{y} - y)^2$, the cost function for regression.)

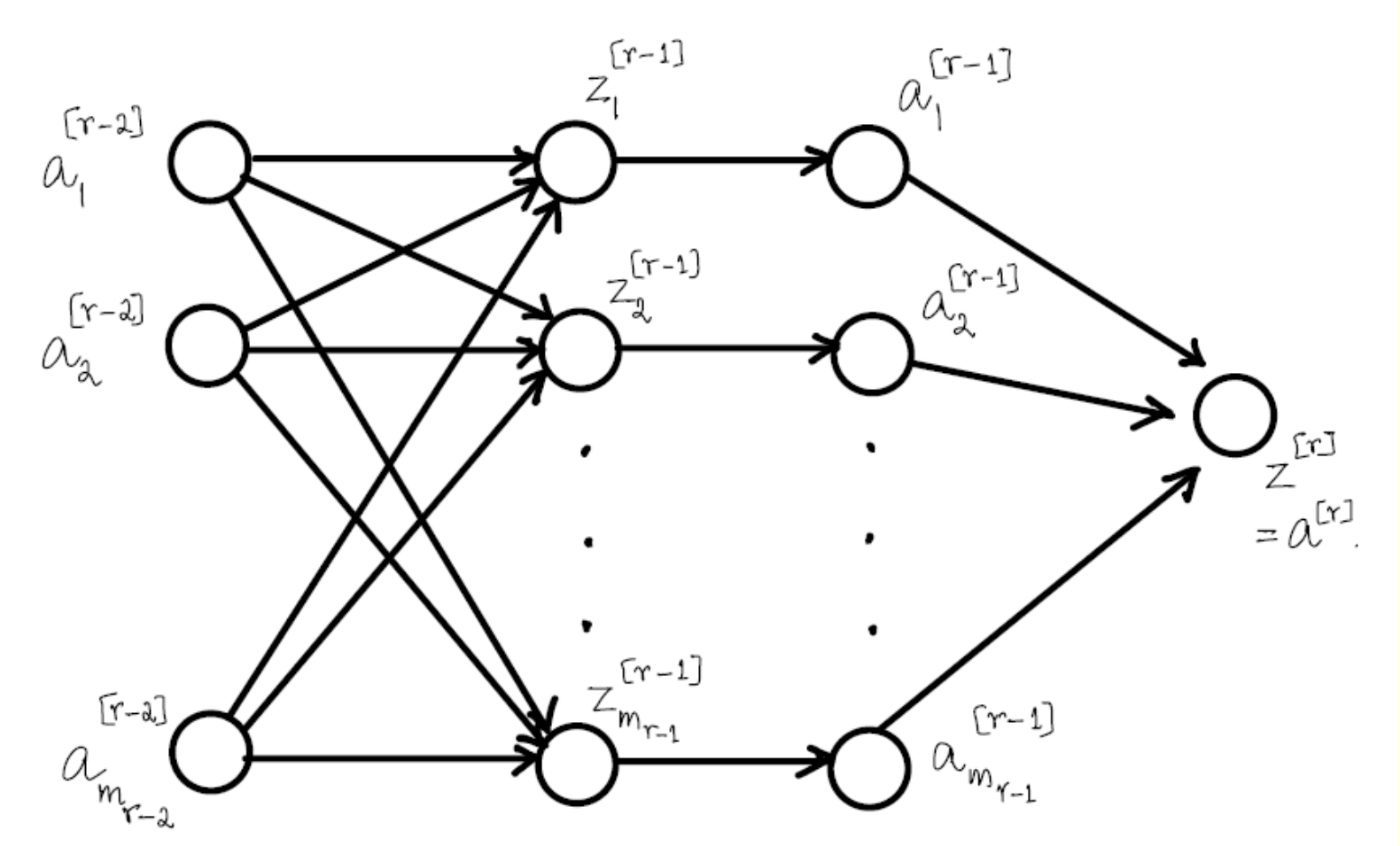


Multi-Layer Fully Connected Neural Network

A multi-layer fully connected neural network (also called a multi-layer perceptron, or MLP) is a type of feedforward artificial neural network where:

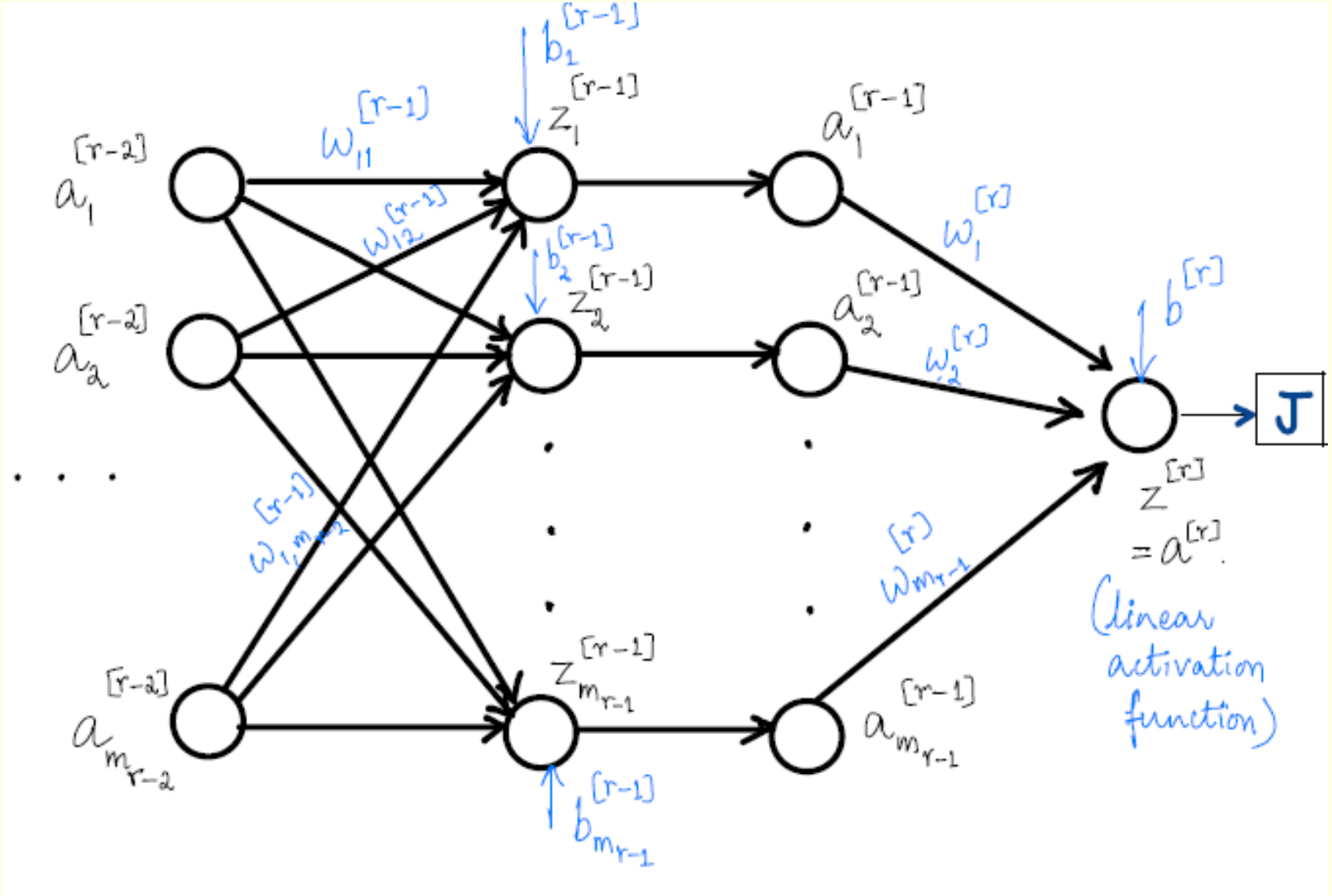
- there is one input layer, at least one hidden layer and one output layer.
- each layer is made up of neurons (nodes).
- every neuron in one layer is connected to every neuron in the next layer.
- these connections have weights that are learned during training.

A visualization of the last three layers of a fully-connected neural network (for regression) with r layers



m_k: Number of neurons in layer k

Visualization along with weights, biases and cost



- The derivative of the cost (loss) with respect to each weight is computed in the same manner and is used to update the corresponding weight.

- Vectorization is used at each layer. The computations during the forward pass take place as matrix-vector multiplications and vector additions. The computations during the backward pass take place in the form of vectorized gradient descent.

- Multiple epochs (iterations) of forward propagation and backward propagation are run to train the neural network. The training stops when the weights converge/ the neural network has learnt a sufficiently good model (loss is sufficiently small).